

Amazon-Like Marketplace Database Architecture

Table of Contents

1. [Overview](#)
 2. [Requirements](#)
 3. [Capacity Estimation](#)
 4. [Schema Design](#)
 5. [ASCII ER Diagram](#)
 6. [Indexing Strategy](#)
 7. [Query Patterns](#)
 8. [Scaling Strategy](#)
 9. [Tradeoffs](#)
 10. [Interview Discussion Points](#)
 11. [Common Interview Follow-ups](#)
 12. [Performance Considerations](#)
 13. [Cross-References](#)
-

Overview

This document models a large-scale marketplace platform similar to Amazon, covering the full lifecycle from product listing to delivery and review. The schema is designed for PostgreSQL and reflects real production considerations: normalization for consistency, denormalization for read performance, and extension points for future growth.

Requirements

Functional Requirements

- Users can register as buyers and/or sellers
- Sellers list products with multiple variants (size, color, etc.)
- Buyers browse, search, and purchase products
- Inventory is tracked at the warehouse/seller level
- Orders flow through a defined state machine (placed → confirmed → shipped → delivered → closed)
- Multiple payment methods: card, bank transfer, wallet
- Buyers leave reviews with ratings after receiving items
- Shipping is tracked with carrier integration
- Sellers receive payouts after order completion

Non-Functional Requirements

- 300M+ registered users globally
 - 2M+ sellers
 - 500M+ product listings
 - 3M+ orders per day at peak
 - Sub-100ms latency for product detail pages
 - 99.99% uptime for checkout and payment flows
 - Data retained for 7 years for regulatory compliance
 - Multi-region deployment
-

Capacity Estimation

Metric	Estimate
Registered users	300M
Active sellers	2M
Product listings	500M
Orders per day	3M
Order items per order (avg)	2.5
Reviews per day	500K
Payments per day	3M
Shipments per day	2.5M

Storage Estimates

Table	Rows	Row Size	Annual Growth
users	300M	500B	+30M rows/yr
products	500M	2KB	+50M rows/yr
inventory	600M	200B	varies
orders	1.1B (historical)	300B	+1.1B rows/yr
order_items	2.7B	200B	+2.7B rows/yr
payments	1.1B	400B	+1.1B rows/yr
reviews	400M	1KB	+180M rows/yr
shipments	1B	300B	+900M rows/yr

Total active storage estimate: ~8TB for hot data, ~80TB including historical

Read/Write ratio: 90:10 for product browsing, 50:50 for checkout

Schema Design

```
-- =====
-- EXTENSIONS
-- =====
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";           -- trigram for full-text search
CREATE EXTENSION IF NOT EXISTS "btree_gin";

-- =====
-- ENUM TYPES
```

```

-- =====
CREATE TYPE user_status AS ENUM ('active', 'suspended', 'closed');
CREATE TYPE order_status AS ENUM (
    'pending', 'confirmed', 'processing',
    'shipped', 'delivered', 'cancelled', 'refunded'
);
CREATE TYPE payment_status AS ENUM (
    'pending', 'authorized', 'captured',
    'failed', 'refunded', 'disputed'
);
CREATE TYPE payment_method AS ENUM (
    'credit_card', 'debit_card', 'bank_transfer',
    'wallet', 'buy_now_pay_later', 'gift_card'
);
CREATE TYPE shipment_status AS ENUM (
    'label_created', 'picked_up', 'in_transit',
    'out_for_delivery', 'delivered', 'attempted',
    'returned', 'lost'
);
CREATE TYPE review_status AS ENUM ('pending', 'published', 'removed');

-- =====
-- USERS
-- =====
CREATE TABLE users (
    user_id        UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    email          VARCHAR(320)  NOT NULL,
    email_verified BOOLEAN      NOT NULL DEFAULT FALSE,
    phone          VARCHAR(30),
    phone_verified BOOLEAN      NOT NULL DEFAULT FALSE,
    full_name      VARCHAR(200)  NOT NULL,
    display_name   VARCHAR(100),
    password_hash  VARCHAR(255)  NOT NULL,
    status         user_status   NOT NULL DEFAULT 'active',
    is_seller      BOOLEAN      NOT NULL DEFAULT FALSE,
    locale         VARCHAR(10)   NOT NULL DEFAULT 'en-US',
    timezone       VARCHAR(60)   NOT NULL DEFAULT 'UTC',
    created_at     TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at     TIMESTAMPTZ   NOT NULL DEFAULT now(),
    last_login_at  TIMESTAMPTZ,
    metadata       JSONB,
    CONSTRAINT uq_users_email UNIQUE (email)
);

CREATE TABLE user_addresses (
    address_id     UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id        UUID          NOT NULL REFERENCES users(user_id) ON DELETE
CASCADE,
    label         VARCHAR(50)    NOT NULL DEFAULT 'Home',
    full_name      VARCHAR(200)  NOT NULL,
    line1         VARCHAR(255)   NOT NULL,
    line2         VARCHAR(255),

```

```

    city            VARCHAR(100)    NOT NULL,
    state           VARCHAR(100),
    postal_code     VARCHAR(20)     NOT NULL,
    country_code    CHAR(2)         NOT NULL,
    is_default      BOOLEAN         NOT NULL DEFAULT FALSE,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

-- =====
-- SELLERS
-- =====

CREATE TABLE sellers (
    seller_id       UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id         UUID            NOT NULL REFERENCES users(user_id),
    store_name      VARCHAR(200)    NOT NULL,
    store_slug      VARCHAR(200)    NOT NULL,
    description     TEXT,
    logo_url        VARCHAR(500),
    rating          NUMERIC(3,2),
    review_count    INT             NOT NULL DEFAULT 0,
    is_verified     BOOLEAN         NOT NULL DEFAULT FALSE,
    is_active       BOOLEAN         NOT NULL DEFAULT TRUE,
    commission_rate NUMERIC(5,4)    NOT NULL DEFAULT 0.15, -- 15% default
    country_code    CHAR(2)         NOT NULL,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uq_sellers_slug UNIQUE (store_slug),
    CONSTRAINT uq_sellers_user UNIQUE (user_id)
);

-- =====
-- PRODUCT CATALOG
-- =====

CREATE TABLE categories (
    category_id     SERIAL          PRIMARY KEY,
    parent_id       INT             REFERENCES categories(category_id),
    name            VARCHAR(200)    NOT NULL,
    slug            VARCHAR(200)    NOT NULL,
    depth           SMALLINT        NOT NULL DEFAULT 0,
    path            LTREE,          -- requires ltree extension for hierarchical
queries
    is_active       BOOLEAN         NOT NULL DEFAULT TRUE,
    CONSTRAINT uq_categories_slug UNIQUE (slug)
);

CREATE TABLE products (
    product_id      UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    seller_id       UUID            NOT NULL REFERENCES sellers(seller_id),
    category_id     INT             NOT NULL REFERENCES categories(category_id),
    title           VARCHAR(500)    NOT NULL,
    slug            VARCHAR(500)    NOT NULL,
    description     TEXT,

```

```

    brand            VARCHAR(200),
    model_number     VARCHAR(200),
    is_active        BOOLEAN          NOT NULL DEFAULT TRUE,
    is_featured      BOOLEAN          NOT NULL DEFAULT FALSE,
    avg_rating       NUMERIC(3,2),
    review_count     INT              NOT NULL DEFAULT 0,
    search_vector     TSVECTOR,        -- for full-text search
    created_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    CONSTRAINT uq_products_slug UNIQUE (seller_id, slug)
);

CREATE TABLE product_variants (
    variant_id       UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    product_id       UUID              NOT NULL REFERENCES products(product_id) ON
DELETE CASCADE,
    sku              VARCHAR(200)      NOT NULL,
    title            VARCHAR(300),     -- e.g., "Red / Large"
    attributes       JSONB             NOT NULL DEFAULT '{}', --
{"color": "red", "size": "L"}
    price            NUMERIC(12,2)     NOT NULL,
    compare_price    NUMERIC(12,2),
    cost_price       NUMERIC(12,2),
    weight_grams     INT,
    is_active        BOOLEAN          NOT NULL DEFAULT TRUE,
    created_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    CONSTRAINT uq_product_variants_sku UNIQUE (sku),
    CONSTRAINT chk_variant_price CHECK (price > 0)
);

CREATE TABLE product_images (
    image_id         UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    product_id       UUID              NOT NULL REFERENCES products(product_id) ON
DELETE CASCADE,
    variant_id       UUID              REFERENCES product_variants(variant_id) ON
DELETE SET NULL,
    url              VARCHAR(1000)     NOT NULL,
    alt_text         VARCHAR(500),
    sort_order       SMALLINT          NOT NULL DEFAULT 0,
    is_primary       BOOLEAN          NOT NULL DEFAULT FALSE
);

-- =====
-- INVENTORY
-- =====

CREATE TABLE warehouses (
    warehouse_id     UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    name             VARCHAR(200)      NOT NULL,
    code             VARCHAR(20)       NOT NULL,
    address_line1     VARCHAR(255)     NOT NULL,
    city             VARCHAR(100)      NOT NULL,
    state            VARCHAR(100),

```

```

postal_code    VARCHAR(20)    NOT NULL,
country_code   CHAR(2)        NOT NULL,
is_active      BOOLEAN        NOT NULL DEFAULT TRUE,
CONSTRAINT uq_warehouses_code UNIQUE (code)
);

CREATE TABLE inventory (
    inventory_id    UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
    variant_id      UUID        NOT NULL REFERENCES
product_variants(variant_id),
    warehouse_id    UUID        NOT NULL REFERENCES warehouses(warehouse_id),
    quantity_on_hand INT        NOT NULL DEFAULT 0,
    quantity_reserved INT       NOT NULL DEFAULT 0,
    quantity_available INT      GENERATED ALWAYS AS (quantity_on_hand -
quantity_reserved) STORED,
    reorder_point   INT         NOT NULL DEFAULT 10,
    reorder_qty     INT         NOT NULL DEFAULT 50,
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_inventory UNIQUE (variant_id, warehouse_id),
    CONSTRAINT chk_inventory_non_negative CHECK (quantity_on_hand >= 0),
    CONSTRAINT chk_inventory_reserved CHECK (quantity_reserved >= 0)
);

CREATE TABLE inventory_transactions (
    txn_id          UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
    inventory_id     UUID        NOT NULL REFERENCES inventory(inventory_id),
    delta           INT         NOT NULL,    -- positive = in, negative = out
    reason          VARCHAR(100) NOT NULL,    -- 'order', 'restock', 'adjustment',
'return'
    reference_id     UUID,        -- order_id, return_id, etc.
    created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    created_by       UUID        REFERENCES users(user_id)
);

-- =====
-- ORDERS
-- =====

CREATE TABLE orders (
    order_id        UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
    order_number     VARCHAR(30) NOT NULL,
    buyer_id        UUID        NOT NULL REFERENCES users(user_id),
    status          order_status NOT NULL DEFAULT 'pending',
    currency        CHAR(3)     NOT NULL DEFAULT 'USD',
    subtotal        NUMERIC(14,2) NOT NULL,
    discount_total   NUMERIC(14,2) NOT NULL DEFAULT 0,
    shipping_total   NUMERIC(14,2) NOT NULL DEFAULT 0,
    tax_total       NUMERIC(14,2) NOT NULL DEFAULT 0,
    grand_total     NUMERIC(14,2) NOT NULL,
    shipping_address_id UUID     REFERENCES user_addresses(address_id),
    notes          TEXT,
    placed_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    confirmed_at   TIMESTAMPTZ,

```

```

        shipped_at      TIMESTAMPTZ,
        delivered_at   TIMESTAMPTZ,
        cancelled_at   TIMESTAMPTZ,
        updated_at     TIMESTAMPTZ      NOT NULL DEFAULT now(),
        CONSTRAINT uq_orders_number UNIQUE (order_number),
        CONSTRAINT chk_order_grand_total CHECK (grand_total >= 0)
    ) PARTITION BY RANGE (placed_at);

-- Monthly partitions (automate with pg_partman in production)
CREATE TABLE orders_2024_01 PARTITION OF orders
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
CREATE TABLE orders_2024_02 PARTITION OF orders
    FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');
-- ... additional partitions created programmatically

CREATE TABLE order_items (
    item_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    order_id         UUID          NOT NULL REFERENCES orders(order_id) ON DELETE
CASCADE,
    variant_id       UUID          NOT NULL REFERENCES
product_variants(variant_id),
    seller_id        UUID          NOT NULL REFERENCES sellers(seller_id),
    quantity         SMALLINT      NOT NULL,
    unit_price       NUMERIC(12,2) NOT NULL,
    discount         NUMERIC(12,2) NOT NULL DEFAULT 0,
    total_price      NUMERIC(12,2) NOT NULL,
    status           order_status NOT NULL DEFAULT 'pending',
    CONSTRAINT chk_item_quantity CHECK (quantity > 0)
);

-- =====
-- PAYMENTS
-- =====

CREATE TABLE payment_methods_stored (
    stored_pm_id     UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id          UUID          NOT NULL REFERENCES users(user_id),
    method           payment_method NOT NULL,
    provider         VARCHAR(50)   NOT NULL,    -- 'stripe', 'braintree', etc.
    token            VARCHAR(500)  NOT NULL,    -- tokenized reference
    last_four        CHAR(4),
    expiry_month     SMALLINT,
    expiry_year      SMALLINT,
    is_default       BOOLEAN       NOT NULL DEFAULT FALSE,
    created_at       TIMESTAMPTZ   NOT NULL DEFAULT now()
);

CREATE TABLE payments (
    payment_id       UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    order_id         UUID          NOT NULL REFERENCES orders(order_id),
    stored_pm_id     UUID          REFERENCES payment_methods_stored(stored_pm_id),
    method           payment_method NOT NULL,
    status           payment_status NOT NULL DEFAULT 'pending',

```

```

    amount          NUMERIC(14,2)    NOT NULL,
    currency         CHAR(3)          NOT NULL DEFAULT 'USD',
    provider         VARCHAR(50)      NOT NULL,
    provider_txn_id  VARCHAR(200),
    authorized_at    TIMESTAMPTZ,
    captured_at      TIMESTAMPTZ,
    failed_at        TIMESTAMPTZ,
    failure_reason   TEXT,
    metadata         JSONB,
    created_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ      NOT NULL DEFAULT now()
);

CREATE TABLE refunds (
    refund_id        UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    payment_id       UUID              NOT NULL REFERENCES payments(payment_id),
    order_id         UUID              NOT NULL REFERENCES orders(order_id),
    amount           NUMERIC(14,2)    NOT NULL,
    reason           TEXT,
    status           payment_status   NOT NULL DEFAULT 'pending',
    provider_refund_id VARCHAR(200),
    created_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    processed_at     TIMESTAMPTZ
);

-- =====
-- SHIPPING
-- =====

CREATE TABLE carriers (
    carrier_id       SERIAL            PRIMARY KEY,
    name             VARCHAR(100)      NOT NULL,
    code            VARCHAR(20)        NOT NULL,
    tracking_url      VARCHAR(500),
    CONSTRAINT uq_carriers_code UNIQUE (code)
);

CREATE TABLE shipments (
    shipment_id      UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    order_id         UUID              NOT NULL REFERENCES orders(order_id),
    carrier_id       INT               REFERENCES carriers(carrier_id),
    tracking_number   VARCHAR(100),
    status           shipment_status   NOT NULL DEFAULT 'label_created',
    shipped_from      UUID              REFERENCES warehouses(warehouse_id),
    estimated_delivery DATE,
    actual_delivery   TIMESTAMPTZ,
    created_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ      NOT NULL DEFAULT now()
);

CREATE TABLE shipment_events (
    event_id         UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    shipment_id      UUID              NOT NULL REFERENCES shipments(shipment_id),

```

```

        status            shipment_status NOT NULL,
        location           VARCHAR(300),
        description        TEXT,
        event_time         TIMESTAMPTZ    NOT NULL,
        created_at         TIMESTAMPTZ    NOT NULL DEFAULT now()
    );

-- =====
-- REVIEWS
-- =====

CREATE TABLE reviews (
    review_id            UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    product_id           UUID            NOT NULL REFERENCES products(product_id),
    variant_id           UUID            REFERENCES product_variants(variant_id),
    order_item_id        UUID            REFERENCES order_items(item_id),
    reviewer_id          UUID            NOT NULL REFERENCES users(user_id),
    rating               SMALLINT        NOT NULL,
    title                VARCHAR(300),
    body                 TEXT,
    status               review_status  NOT NULL DEFAULT 'pending',
    helpful_votes        INT             NOT NULL DEFAULT 0,
    verified_purchase     BOOLEAN        NOT NULL DEFAULT FALSE,
    created_at           TIMESTAMPTZ    NOT NULL DEFAULT now(),
    updated_at           TIMESTAMPTZ    NOT NULL DEFAULT now(),
    CONSTRAINT chk_review_rating CHECK (rating BETWEEN 1 AND 5),
    CONSTRAINT uq_review_per_item UNIQUE (order_item_id, reviewer_id)
);

CREATE TABLE review_media (
    media_id            UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    review_id           UUID            NOT NULL REFERENCES reviews(review_id) ON DELETE
CASCADE,
    url                 VARCHAR(1000)   NOT NULL,
    media_type          VARCHAR(20)     NOT NULL DEFAULT 'image'
);

```

ASCII ER Diagram

```

+-----+      +-----+      +-----+
|  users  |1-----N|user_addr |      | categories|
+-----+      +-----+      +-----+
      |1                      |1
      |N                      |N
+-----+      +-----+      +-----+
| sellers |1-----N| products |N-----1| product  |
+-----+      +-----+      | variants |
      |                      |1
      |                      |N
      |      +-----+      |N
      |      |prod_imgs |      +-----+

```

```

      |
      |
      |
      |
      |
+-----+
| orders |1-----N|order_item|
+-----+
      |1
      |N
      |
+-----+
| payments |
+-----+
      |1
      |N
+-----+
| refunds |
+-----+

+-----+
| orders |1-----N| shipments |1-----N shipment_events
+-----+

+-----+
| products |1-----N| reviews |
+-----+

```

Indexing Strategy

```

-- Users: login and lookup
CREATE INDEX idx_users_email      ON users (email);
CREATE INDEX idx_users_status    ON users (status) WHERE status != 'active';

-- Products: search and browse
CREATE INDEX idx_products_seller  ON products (seller_id);
CREATE INDEX idx_products_category ON products (category_id);
CREATE INDEX idx_products_search  ON products USING GIN (search_vector);
CREATE INDEX idx_products_rating  ON products (avg_rating DESC NULLS LAST)
    WHERE is_active = TRUE;

-- Product variants: SKU lookup and price filtering
CREATE INDEX idx_variants_product ON product_variants (product_id);
CREATE INDEX idx_variants_sku     ON product_variants (sku);
CREATE INDEX idx_variants_price   ON product_variants (price);

-- Inventory: availability checks (hot path)
CREATE INDEX idx_inventory_variant ON inventory (variant_id)
    WHERE quantity_available > 0;

```

```

-- Orders: buyer history and status tracking
CREATE INDEX idx_orders_buyer      ON orders (buyer_id, placed_at DESC);
CREATE INDEX idx_orders_status     ON orders (status) WHERE status NOT IN
('delivered','cancelled');

-- Order items: seller fulfillment view
CREATE INDEX idx_order_items_seller ON order_items (seller_id, status);
CREATE INDEX idx_order_items_order  ON order_items (order_id);

-- Payments: reconciliation
CREATE INDEX idx_payments_order     ON payments (order_id);
CREATE INDEX idx_payments_status    ON payments (status, created_at)
    WHERE status IN ('pending','authorized');

-- Shipments: tracking lookup
CREATE INDEX idx_shipments_order    ON shipments (order_id);
CREATE INDEX idx_shipments_tracking ON shipments (tracking_number);

-- Reviews: product page aggregation
CREATE INDEX idx_reviews_product    ON reviews (product_id, status, created_at
DESC);
CREATE INDEX idx_reviews_reviewer   ON reviews (reviewer_id);

```

Query Patterns

Q1 — Product Detail Page (most frequent read)

```

SELECT
  p.product_id, p.title, p.description, p.avg_rating, p.review_count,
  s.store_name, s.rating AS seller_rating,
  json_agg(DISTINCT jsonb_build_object(
    'variant_id', pv.variant_id,
    'sku', pv.sku,
    'title', pv.title,
    'price', pv.price,
    'attributes', pv.attributes,
    'available', COALESCE(SUM(i.quantity_available), 0)
  )) AS variants,
  json_agg(DISTINCT jsonb_build_object('url', pi.url, 'primary', pi.is_primary)
    ORDER BY (pi.is_primary) DESC) AS images
FROM products p
JOIN sellers s ON s.seller_id = p.seller_id
JOIN product_variants pv ON pv.product_id = p.product_id AND pv.is_active
LEFT JOIN inventory i ON i.variant_id = pv.variant_id
LEFT JOIN product_images pi ON pi.product_id = p.product_id
WHERE p.product_id = $1 AND p.is_active
GROUP BY p.product_id, s.seller_id;

```

Q2 — Category Browse with Filters

```

SELECT
    p.product_id, p.title, p.avg_rating, p.review_count,
    MIN(pv.price) AS min_price,
    MAX(pv.price) AS max_price
FROM products p
JOIN categories c ON c.category_id = p.category_id
JOIN product_variants pv ON pv.product_id = p.product_id AND pv.is_active
WHERE c.path <@ text2ltree($1)  -- ltree hierarchy query
    AND p.is_active
    AND pv.price BETWEEN $2 AND $3
GROUP BY p.product_id
ORDER BY p.avg_rating DESC NULLS LAST
LIMIT 50 OFFSET $4;

```

Q3 — Full-Text Product Search

```

SELECT
    p.product_id, p.title, p.avg_rating,
    ts_rank(p.search_vector, query) AS rank,
    MIN(pv.price) AS min_price
FROM products p,
    to_tsquery('english', $1) query
JOIN product_variants pv ON pv.product_id = p.product_id AND pv.is_active
WHERE p.search_vector @@ query
    AND p.is_active
ORDER BY rank DESC, p.avg_rating DESC NULLS LAST
LIMIT 50;

```

Q4 — Place an Order (transactional)

```

BEGIN;

-- Reserve inventory
UPDATE inventory
SET quantity_reserved = quantity_reserved + $qty
WHERE variant_id = $variant_id
    AND quantity_available >= $qty;

-- Fail if no rows updated
-- (application checks rowcount)

INSERT INTO orders (order_number, buyer_id, subtotal, grand_total, ...)
VALUES ($order_number, $buyer_id, $subtotal, $grand_total, ...)
RETURNING order_id;

INSERT INTO order_items (order_id, variant_id, seller_id, quantity, unit_price,
total_price)
VALUES ($order_id, $variant_id, $seller_id, $qty, $unit_price, $total_price);

```

```
COMMIT;
```

Q5 — Buyer Order History

```
SELECT
    o.order_id, o.order_number, o.status, o.grand_total, o.placed_at,
    COUNT(oi.item_id) AS item_count,
    json_agg(jsonb_build_object(
        'title', p.title,
        'image', (SELECT url FROM product_images WHERE product_id = p.product_id AND
is_primary LIMIT 1),
        'quantity', oi.quantity,
        'price', oi.total_price
    )) AS items
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN product_variants pv ON pv.variant_id = oi.variant_id
JOIN products p ON p.product_id = pv.product_id
WHERE o.buyer_id = $1
GROUP BY o.order_id
ORDER BY o.placed_at DESC
LIMIT 20 OFFSET $2;
```

Q6 — Seller Dashboard: Pending Fulfillments

```
SELECT
    o.order_id, o.order_number, o.placed_at,
    oi.item_id, oi.quantity, oi.unit_price,
    pv.sku, p.title,
    u.full_name AS buyer_name,
    ua.line1, ua.city, ua.country_code
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN product_variants pv ON pv.variant_id = oi.variant_id
JOIN products p ON p.product_id = pv.product_id
JOIN users u ON u.user_id = o.buyer_id
JOIN user_addresses ua ON ua.address_id = o.shipping_address_id
WHERE oi.seller_id = $1
    AND oi.status = 'confirmed'
ORDER BY o.placed_at ASC
LIMIT 100;
```

Q7 — Product Reviews Summary

```
SELECT
    rating,
    COUNT(*) AS count,
```

```

ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 1) AS pct
FROM reviews
WHERE product_id = $1 AND status = 'published'
GROUP BY rating
ORDER BY rating DESC;

```

Q8 — Inventory Low-Stock Alert

```

SELECT
    w.name AS warehouse, pv.sku, p.title,
    i.quantity_on_hand, i.quantity_reserved, i.quantity_available,
    i.reorder_point
FROM inventory i
JOIN product_variants pv ON pv.variant_id = i.variant_id
JOIN products p ON p.product_id = pv.product_id
JOIN warehouses w ON w.warehouse_id = i.warehouse_id
WHERE i.quantity_available <= i.reorder_point
    AND p.is_active AND pv.is_active
ORDER BY i.quantity_available ASC;

```

Q9 — Payment Reconciliation Report

```

SELECT
    DATE_TRUNC('day', p.created_at) AS day,
    p.method,
    p.status,
    COUNT(*) AS txn_count,
    SUM(p.amount) AS total_amount,
    p.currency
FROM payments p
WHERE p.created_at >= NOW() - INTERVAL '30 days'
GROUP BY 1, 2, 3, 6
ORDER BY 1 DESC, 4 DESC;

```

Q10 — Shipment Tracking History

```

SELECT
    s.shipment_id, s.tracking_number,
    c.name AS carrier,
    s.status AS current_status,
    s.estimated_delivery,
    json_agg(jsonb_build_object(
        'status', se.status,
        'location', se.location,
        'description', se.description,
        'time', se.event_time
    ) ORDER BY se.event_time DESC) AS events
FROM shipments s

```

```
JOIN orders o ON o.order_id = s.order_id
LEFT JOIN carriers c ON c.carrier_id = s.carrier_id
LEFT JOIN shipment_events se ON se.shipment_id = s.shipment_id
WHERE o.buyer_id = $1
      AND s.shipment_id = $2
GROUP BY s.shipment_id, c.name;
```

Scaling Strategy

Phase 1: Single Primary (0–10M users)

- Single PostgreSQL instance with read replica
- Connection pooling via PgBouncer (pool_mode=transaction)
- Caching layer: Redis for product pages, sessions, inventory counts
- CDN for product images

Phase 2: Functional Sharding (10M–100M users)

At this scale, partition tables by time and separate read-heavy from write-heavy workloads:

- Orders table: range-partition by `placed_at` (monthly), archive to cold storage
- inventory reads: dedicated read replicas + Redis write-through cache
- Product search: offload to Elasticsearch / OpenSearch, sync via logical replication
- User sessions: move entirely to Redis cluster

Phase 3: Horizontal Scaling (100M+ users)

- Shard orders and order_items by `buyer_id % N` (keeps buyer queries local)
- Shard inventory by `warehouse_id` (geographic shards align with fulfillment)
- Separate microservices with dedicated databases: Catalog DB, Order DB, Payment DB
- CQRS: write to PostgreSQL, project to denormalized read stores (Cassandra / DynamoDB for high-volume reads)
- Introduce Kafka for cross-service events (OrderPlaced → InventoryReserved → ShipmentCreated)

Caching Strategy

```
Browser → CDN (product images, static content)
          → API Gateway cache (product pages, 60s TTL)
          → Redis (inventory counts, user sessions, cart)
          → PostgreSQL (authoritative source)
```

Tradeoffs

Decision	Chosen	Alternative	Why
UUID primary keys	UUID v4	BIGSERIAL	Better for distributed systems; avoids sequential scan of index hotspot
Partitioned orders	Range by date	No partition	Enables fast archive and pruning of old data; partition pruning speeds up recent-orders queries

JSON attributes on variants	JSONB column	EAV tables	Flexible for arbitrary attributes; JSONB is queryable and indexed
Denormalized avg_rating	Stored on products	Always compute	Avoids COUNT(*) on millions of reviews per product page load
Inventory as row	Single row per (variant, warehouse)	Event-sourced ledger	Simpler for most queries; use inventory_transactions for full audit trail

Interview Discussion Points

- 1. Why UUID over BIGINT for primary keys?** UUIDs prevent enumeration attacks (security), enable client-generated IDs (less round trips), and work across distributed inserts without a central sequence. The downside is index bloat — use UUIDv7 (time-ordered) in production to mitigate.
- 2. How do you handle concurrent inventory updates?** The `UPDATE inventory SET quantity_reserved = quantity_reserved + $qty WHERE quantity_available >= $qty` is atomic at the row level. Under high contention (flash sales), use `SELECT FOR UPDATE SKIP LOCKED` or a Redis-based counter with periodic reconciliation.
- 3. How is the search_vector maintained?** Use a trigger that calls `to_tsvector('english', title || ' ' || description)` on insert/update, or update it via a background job. For scale, Elasticsearch is preferred.
- 4. Why partition orders by date?** 90% of queries hit recent orders (last 90 days). Partitioning allows PostgreSQL to skip historical partitions entirely via partition pruning, reducing I/O dramatically.

Common Interview Follow-ups

Q: How would you handle a flash sale where 100K users try to buy the last unit? A: Use an optimistic locking pattern with `quantity_available >= 1` in the WHERE clause. The first successful UPDATE wins. Alternatively, use a Redis counter decremented atomically with DECR — only proceed to PostgreSQL for the actual order when Redis counter > 0.

Q: How do you prevent duplicate orders from double-click / network retry? A: Include an idempotency key (client-generated UUID) stored in orders. Use `INSERT ... ON CONFLICT (idempotency_key) DO NOTHING RETURNING order_id`.

Q: What would you change to support 1B users? A: Introduce database per region (active-active with conflict resolution), event-driven inventory updates via Kafka, read-only product catalog replicated to edge locations, and asynchronous order confirmation flow.

Performance Considerations

- Connection pooling:** At 10K RPS, direct connections overwhelm PostgreSQL. PgBouncer in transaction mode reduces connections from thousands to dozens.
- Bulk inserts:** Use `COPY` or multi-row `INSERT` for inventory restocking jobs.
- VACUUM and bloat:** High-churn tables (inventory, payments) require aggressive `autovacuum_vacuum_scale_factor = 0.01`.

- **Prepared statements:** Use parameterized queries for all hot paths; reduces parse overhead by ~30%.
 - **work_mem:** Tune per-session for heavy reporting queries; default 4MB is too low for GROUP BY on millions of rows.
-

Cross-References

- See `04_banking_platform.md` for double-entry payment ledger patterns
- See `21_System_Design/02_ecommerce_system_design.md` for full system architecture
- See `21_System_Design/08_system_design_framework.md` for interview presentation framework
- PostgreSQL partitioning: Chapter 5 of this guide
- Full-text search: Chapter 9 of this guide